A

PROJECT REPORT

ON

Real-Time Speech Noise Reduction on an ESP32-S3 Using Short-Time Spectral Gain Estimation

Submitted For

EC301: Digital Signal Processing

Submitted to

Dr. Shivnarayan Patidar

Associate Professor

SUBMITTED BY

Kartikey Tiwari (24ECE1020)
Karun Raikar (24ECE1021)
Kathan Kuchekar (24ECE1022)

Department of Electronics and Communication Engineering
National Institute of Technology Goa
South Goa, Goa

Source code and firmware:
`https://github.com/kartikeyy12/hushwave-speech-denoiser`

Contents

Abstract 3
Chapter 1. Introduction to Spectral Noise Reduction 3
1.1 Short-time spectral attenuation 3
1.2 Noise tracking and gain rules 3
1.3 Where the tools are used 4
Chapter 2. Problem Statement 4
Chapter 3. Implementation Details 5
Chapter 4. Python and Firmware Programming 5
Chapter 5. Results and Discussion 6
5.1 Accuracy of the system 6
5.2 Input SNR and noise type 6
5.3 Ablations 7
5.4 Time and frequency domain 7
5.5 Real recordings 10
5.6 Microcontroller implementation 10
5.7 Limitations 11
Chapter 6. Conclusion 11
Chapter 7. References 12

Abstract

This project builds a single-channel speech noise reduction system that runs in real time on an ESP32-S3 microcontroller. The computer records the microphone, sends blocks of 16 ms over USB to the board, and plays the denoised blocks that come back. On the board, the signal is analysed with a 32 ms short-time Fourier transform, the noise spectrum is tracked with minima-controlled recursive averaging, and every time-frequency cell is attenuated by a decision-directed Wiener gain. The FFT, the windows, the tracker, the gain rules, the metrics and the serial protocol are written for the project.

The parameters were tuned on a development set and the test set was evaluated once. On 24 held-out LibriSpeech utterances mixed with 11 noise conditions at 5 input SNRs, the mean SNR improvement is 5.44 dB (95 % interval 4.81 to 6.01 dB), against 5.20 dB for tuned spectral subtraction. The advantage is small, the perceptual score (PESQ) is equal, and no method of this family improves STOI. On two own recordings without a clean reference the noise reduction in pauses is modest. The firmware, compiled for a computer and driven through the same serial protocol, agrees with the Python reference to within 1.7 steps of 16-bit audio on 7 test signals. The algorithmic delay is 31.9 ms.

Chapter 1
Introduction to Spectral Noise Reduction

1.1 Short-time spectral attenuation

A microphone records speech together with fans, traffic and other people. With one microphone there is no spatial information, so the two signals are separated by their statistics: speech is sparse in time and frequency, and many noises change slowly. The signal is cut into overlapping frames and transformed with an FFT. A noise power spectrum $\lambda_d(k, m)$ is tracked, and each coefficient $Y_k(m)$ is multiplied by a gain $G_k(m)$ that depends on how far its power rises above the noise:

$$\hat{S}_k(m) = G_k(m)\, Y_k(m), \qquad \gamma_k(m) = \frac{|Y_k(m)|^2}{\lambda_d(k, m)}. \tag{1.1}$$

The enhanced frames are transformed back and added together with overlap.

1.2 Noise tracking and gain rules

The noise cannot be measured while speech is present, so it is tracked. Minimum statistics [3] takes the minimum of the smoothed power over about two seconds. Minima-controlled recursive averaging (MCRA) compares the smoothed power with its running minimum, declares speech present above a threshold, and updates the noise estimate quickly where speech is absent and slowly where it is likely [4]. The Wiener gain $G = \xi/(1+\xi)$ needs the a priori SNR ξ, which the decision-directed estimate obtains from the previous enhanced frame [2]:

$$\hat{\xi}_k(m) = \alpha\, \frac{\hat{A}_k^2(m-1)}{\lambda_d(k, m-1)} + (1-\alpha)\, \max\big(\gamma_k(m) - 1, 0\big). \tag{1.2}$$

A value of α close to one smooths ξ over time and suppresses isolated bursts of residual noise. Spectral subtraction [1] and the plain-SNR Wiener gain serve as comparisons. The frames are windowed and added with overlap as in [5], and the transform is a radix-2 FFT [6].

1.3 Where the tools are used

Use in the project	Tool
Analysis and synthesis	Square-root Hann window, radix-2 FFT, 50 % overlap-add
Noise power estimate	MCRA with a rise-rate limit
Attenuation	Wiener gain, decision-directed a priori SNR
Low-frequency rumble	Zero-phase 80 Hz spectral high-pass
Real-time processing	Block streaming engine in C++ on the ESP32-S3

Chapter 2
Problem Statement

The task is to remove steady background noise from speech in real time: audio is recorded, processed immediately, and the denoised audio is played back. The constraints are: the algorithms are implemented in the project and not called from a library; the processing runs on a microcontroller; there is no learned model; parameters are chosen on development data only; and the test set is evaluated once.

Sampling frequency. Speech carries intelligibility information up to about 8 kHz, so the system samples at 16 kHz. This keeps the fricatives, matches the corpora and the wide-band quality measures, and keeps the cost low. Resampling from a higher rate needs an anti-alias filter: a two-tone test decimated from 48 to 16 kHz folds an 11 kHz tone to 5000 Hz at 0.0 dB without a filter and at -68.1 dB with one. With 16-bit samples the quantisation noise of a full-scale sine is 98.1 dB below the signal, and of a signal 40 dB below full scale 58.1 dB, far below any room noise.

Data. Speech comes from LibriSpeech [9] and noise from the DEMAND database [10], from synthetic generators (white, pink, mains hum, babble, amplitude-modulated noise, sparse transients) and from two own recordings (a fan and a corridor). Development and test utterances come from different speakers (Table 2.1). The test set crosses 24 utterances, 11 noise conditions and input SNRs of -5, 0, 5, 10, 15 dB, which gives 1320 cases.

Set	Speakers	Utterances	Speech (min)
Development (tuning)	4	16	1.4
Test (held out)	6	24	2.1

Table 2.1: Speech used for tuning and for testing.

Chapter 3
Implementation Details

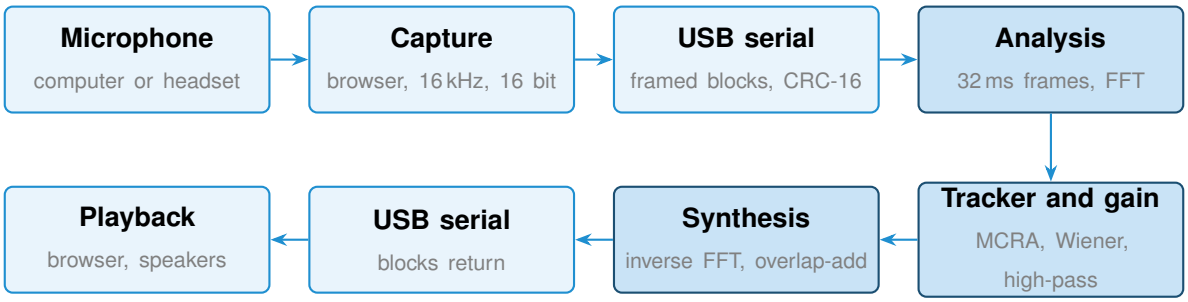


Figure 3.1: Block diagram. The darker boxes run on the ESP32-S3.

- **Acquisition and playback.** A web page captures the microphone at 16 kHz with its own noise suppression, echo cancellation and gain control switched off, and plays the returned audio. It also records the original and the denoised signal as two aligned files. Any computer with Chrome or Edge works.
- **Framing.** Frames of 512 samples (32 ms) with a hop of 256 samples (16 ms), square-root Hann window, so that the analysis and synthesis windows together sum to one. The FFT is radix-2.
- **Noise tracking.** MCRA with $\alpha_s = 0.8$, $\alpha_p = 0.2$, $\alpha_d = 0.95$ and threshold $\delta = 3$. The running minimum covers up to 96 frames (about 1.5 s) in six sub-windows. A rise-rate limit of 3 dB per second keeps the estimate from following speech when the last pause leaves the window.
- **Gain.** Wiener gain with decision-directed SNR ($\alpha = 0.9$) and a floor of $g_{\min} = 0.03$. A zero-phase mask with the magnitude of an 80 Hz second-order Butterworth high-pass multiplies the gain, so that the low-frequency cut adds no phase shift.
- **Streaming.** The engine accepts one hop at a time. A sample is final only after the last frame containing it has been processed, so the output is delayed by $N - 1 = 511$ samples, that is 31.9 ms, independent of how the audio is split into blocks.
- **Link to the board.** Each block of 256 samples travels as one frame of 520 bytes: two sync bytes, type, sequence number, length, 16-bit PCM payload and a CRC-16. The board answers with a frame of 524 bytes that also carries the processing time of the block. Four blocks are in flight at once, so the transmission time hides behind the computation. The parser resynchronises after damaged bytes.

Chapter 4
Python and Firmware Programming

The research code is Python 3.11 with NumPy and SciPy. The FFT, windows, STFT, filter, noise trackers, gain rules, metrics, Welch estimator, statistics and experiment runner are implemented in the project; libraries read and write files, draw plots, and supply the external reference systems

used for comparison (`scipy.signal.wiener`, `noisereduce`) and the perceptual measures (`pesq`, `pystoi`). The firmware is C++ without any library besides the C maths functions: a single-precision port of the tuned engine, whose constants are generated from the tuning file. It uses 31 kB of the 512 kB of RAM of the ESP32-S3. The same sources are compiled for a computer, where they run behind the serial protocol exactly as on the board. The verification step runs 927 automated tests with 98.5 % line coverage of the Python package and passes.

Source code, results and firmware:
`https://github.com/kartikeyy12/hushwave-speech-denoiser`

Chapter 5
Results and Discussion

5.1 Accuracy of the system

Table 5.1 gives the result on the test set (CI: confidence interval). The metrics are read together, because a method that mutes everything scores well on noise reduction and badly on the others. ΔSNR is the improvement against the clean speech, ΔsegSNR the same on speech blocks, NR the noise reduction on noise-only blocks, and MNR the musical-noise ratio (values above one mean more isolated spectral peaks than in the input). Perceptual quality is PESQ in its wide-band form [8] and intelligibility is STOI [11]. Confidence intervals are bootstrap intervals over utterances, and tests are paired Wilcoxon tests with Holm correction [12].

System	ΔSNR (dB) [95 % CI]	ΔsegSNR (dB)	ΔPESQ	ΔSTOI	NR (dB)	MNR
B1 spectral subtraction	5.20 [4.58, 5.76]	3.03	0.24	0.006	9.5	2.16
B2 scipy Wiener	0.52 [0.32, 0.67]	−0.75	−0.04	−0.011	1.4	1.56
B3 noisereduce	−1.93 [−2.22, −1.67]	−2.46	−0.21	−0.035	21.3	5.04
B4 Wiener, plain SNR	4.74 [4.19, 5.23]	2.97	0.17	0.006	7.2	1.98
B5 HushWave	5.44 [4.81, 6.01]	3.54	0.25	0.003	10.0	2.56

Table 5.1: Test set, 24 utterances, means over utterances.

HushWave (B5) has the largest SNR and segmental SNR improvement. The advantage over spectral subtraction is small: the median per-utterance difference is 0.26 dB in ΔSNR and 0.55 dB in ΔsegSNR, both significant after correction (Holm $p < 0.001$). Against the plain-SNR Wiener gain the difference is 0.75 dB, which is what decision-directed estimation adds. PESQ is equal to spectral subtraction (0.25 against 0.24, Holm $p = 0.415$). No method of this family improves STOI. The general-purpose denoiser B3 removes the most noise (21.3 dB) but also the speech: its SNR improvement is negative and it lowers PESQ.

5.2 Input SNR and noise type

B5 has the highest mean ΔSNR at every input SNR (Figure 5.1), from 7.4 dB at −5 dB to 2.9 dB at 15 dB. The gain is largest for steady and narrow-band noise and small for babble. On the transient noise B5 gives −0.2 dB, that is, no benefit, because a minimum-based tracker sees a transient as signal. On babble spectral subtraction is better (1.4 dB against 1.1 dB), since babble

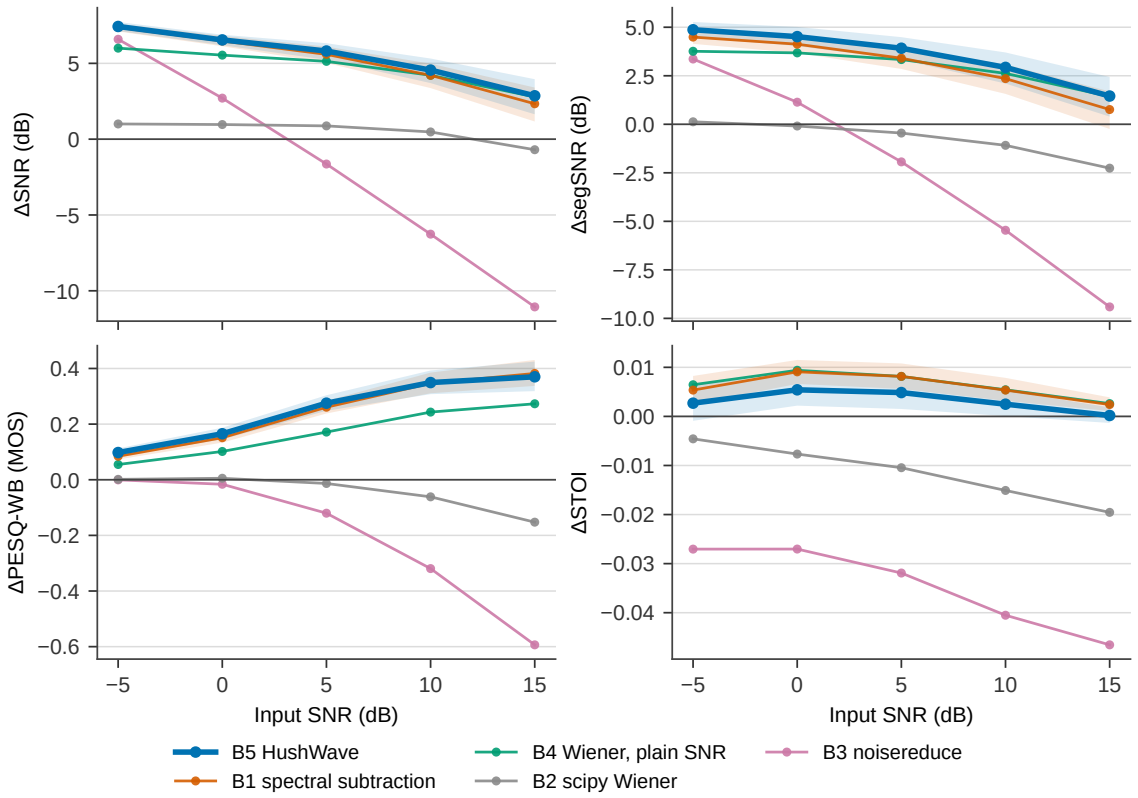


Figure 5.1: Improvement over the noisy input against input SNR, with 95 % intervals for B5 and B1.

has the structure of speech itself (Figure 5.2).

5.3 Ablations

Each ablation changes one component of B5 (Figure 5.3). The causal biquad in place of the zero-phase high-pass costs 4.4 dB, and removing the high-pass costs 1.7 dB. Frames of 16 ms are worse than 32 ms; 64 ms frames are not distinguishable. Two choices made on the development data are not supported on the test data. The minimum-statistics tracker is about 0.1 dB better in ΔSNR than MCRA (Holm $p = 0.011$), and the rise limit gives no significant SNR gain (Holm $p = 0.604$) while lowering the noise reduction from 16.6 dB to 10.0 dB. Neither was changed after the test evaluation.

5.4 Time and frequency domain

Figure 5.4 shows one test case (cafeteria noise at 5 dB input SNR). The output follows the clean envelope, the noise floor between words is lower, and the strongest peaks keep their amplitude. The spectrograms in Figure 5.5 show the broad-band noise floor suppressed in the pauses and between the harmonics, while the harmonic structure of the voiced segments remains. Isolated residual cells in the pauses are the musical noise measured by MNR, and weak high-frequency speech is attenuated together with the noise.

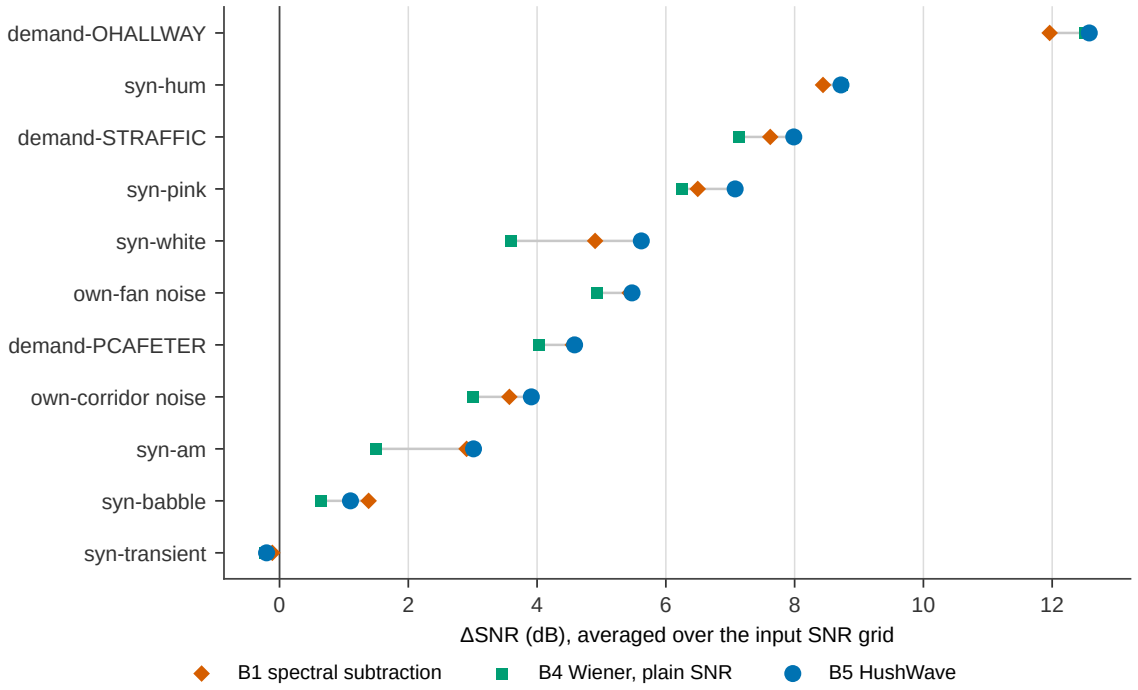


Figure 5.2: ΔSNR per noise condition. B2 and B3 are omitted; they are in Table 5.1.

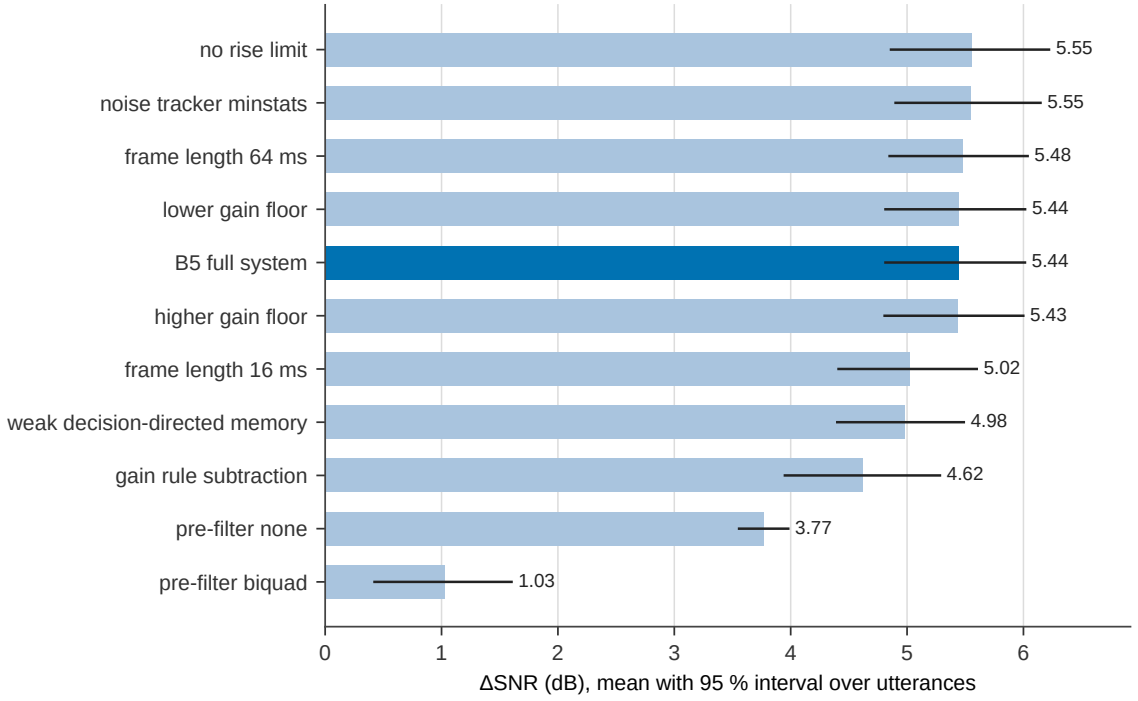


Figure 5.3: ΔSNR of B5 and of each variant with one component changed.

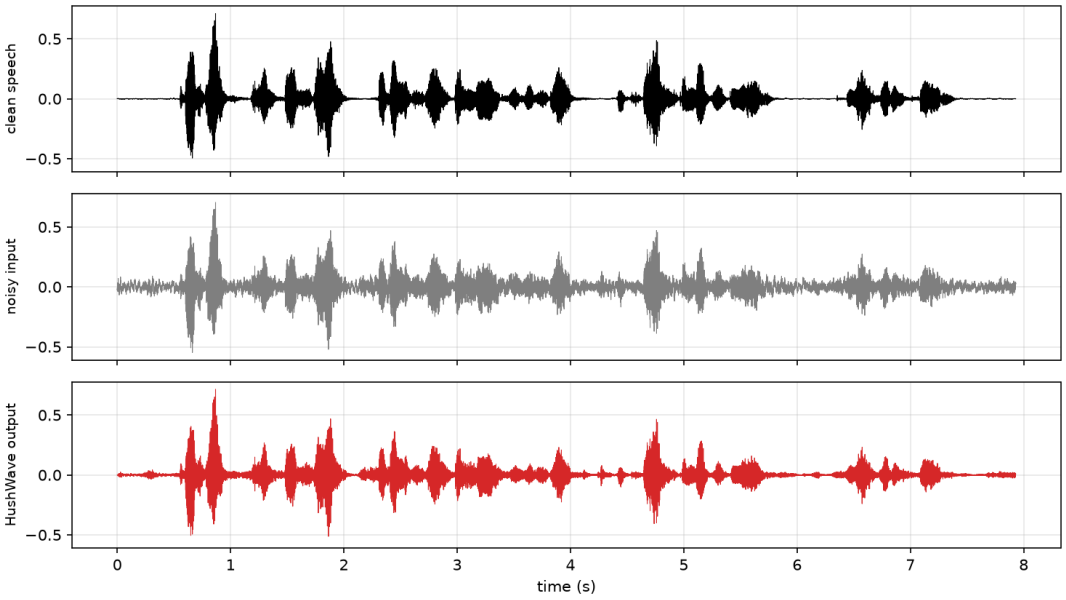


Figure 5.4: Clean speech, noisy input and HushWave output for one test case.

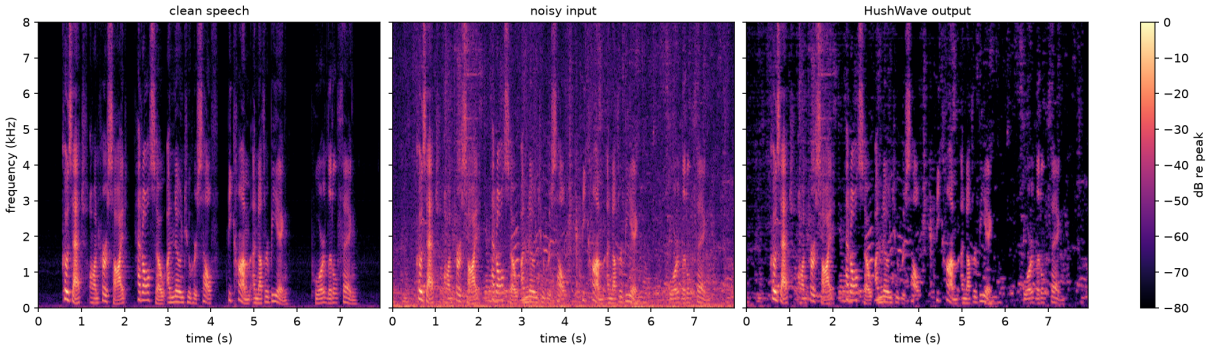


Figure 5.5: Spectrograms of the same case.

5.5 Real recordings

The two own recordings have no clean reference, so the code measures what the system does to pauses and to speech (Table 5.2, Figure 5.6; spectra by Welch's method [7]). This is the weakest result: HushWave reduces the noise in pauses by 2.8 dB (fan) and 1.9 dB (corridor), spectral subtraction by 3.9 dB and 2.1 dB. For mixtures of LibriSpeech with the same noise the reduction is 9.0 dB and 9.2 dB (different definition), which suggests that continuous speech with short pauses limits the trackers. We did not isolate this. B3 removes 15 to 20 dB of pause noise but lowers the speech level by about 10 to 11 dB.

System	NR in pauses (dB)	Pause level (dBFS)	Speech level (dB)	MNR
Speech in fan noise				
B1 spectral subtraction	3.9	−46.1	−0.6	2.77
B2 scipy Wiener	1.5	−43.6	−0.2	2.19
B3 noisereduce	15.1	−57.3	−9.8	6.85
B4 Wiener, plain SNR	2.3	−44.5	−0.1	2.11
B5 HushWave	2.8	−45.0	−0.2	2.87
Speech in corridor noise				
B1 spectral subtraction	2.1	−39.7	−0.3	1.48
B2 scipy Wiener	0.7	−38.3	−0.2	0.99
B3 noisereduce	20.1	−57.7	−11.3	6.39
B4 Wiener, plain SNR	1.8	−39.4	−0.1	1.33
B5 HushWave	1.9	−39.5	−0.2	1.42

Table 5.2: Real recordings. NR: noise reduction in pause blocks. Speech level: change of the energy of speech blocks.

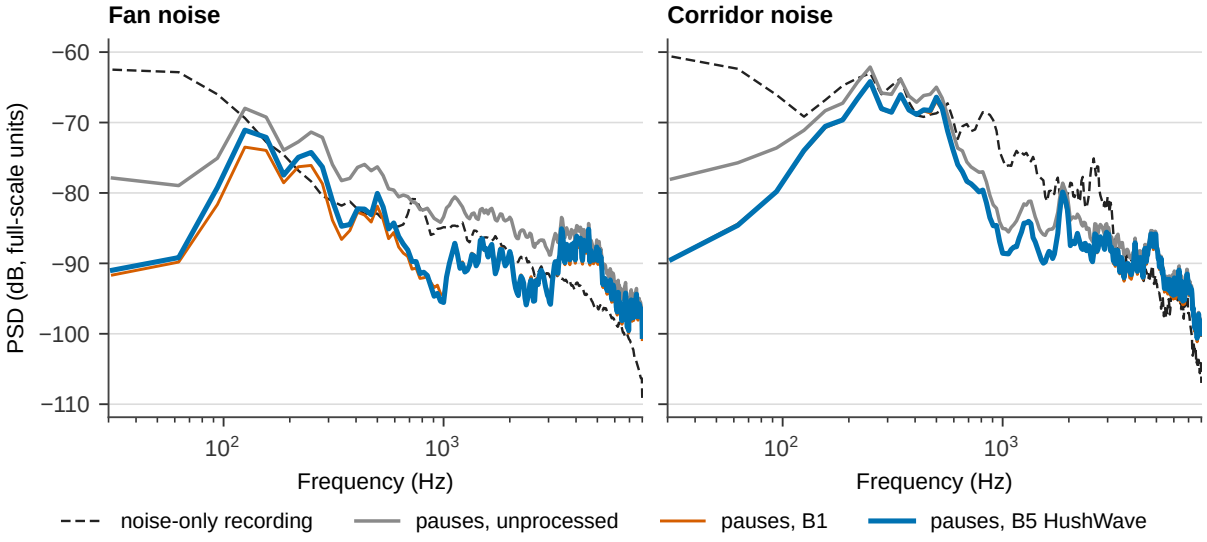


Figure 5.6: Welch power spectra of the pauses before and after processing.

5.6 Microcontroller implementation

The firmware was compared with the Python engine on 7 signals of 12 s (5250 blocks in total), among them silence, near-silent noise, full-scale noise, a clipped tone, impulses and a level

step. Both see the same 16-bit input and both outputs are saturated at 16 bit. Table 5.3 lists the differences in steps of 16-bit audio. The largest is 1.7 steps (-86 dB relative to full scale); a rounding error alone gives up to half a step. On the voiced test signal the firmware output differs from the reference by 76 dB in SNR. The serial link carries 32.5 kB/s towards the board and 32.8 kB/s back, which is 35 % of the capacity at 921600 baud. The firmware measures the processing time of every block and reports it to the interface, which shows the current value while the system runs.

Test signal (12 s each)	Largest difference	RMS difference
voiced bursts in noise	0.50	0.29
silence	0.00	0.00
very quiet noise	0.50	0.29
full-scale noise	0.57	0.29
clipped tone	0.55	0.29
impulses	0.50	0.12
level step	1.67	0.29

Table 5.3: Firmware against the Python engine, in steps of 16-bit audio (one step is 3.05×10^{-5} of full scale).

5.7 Limitations

- The musical-noise ratio of B5 (2.56) is the highest of the classical systems (B1 2.16, B4 1.98).
- There is no improvement on transient noise, and spectral subtraction is better on babble.
- On the real recordings the noise reduction in pauses is no better than that of spectral subtraction.
- Test mixtures are synthetic. The corridor recordings are 48 kHz stereo and the others 44.1 kHz dual mono, which digital mixing makes harmless.
- The own-voice test set has 6 segments, too few for corrected significance tests.
- The transport adds only 16-bit rounding, so the audio quality is set by the algorithm and its limits listed above.

Chapter 6
Conclusion

The project delivers a complete real-time chain: microphone, USB link, an ESP32-S3 that runs the tuned noise reducer, and playback, with a fixed algorithmic delay of 31.9 ms. On held-out data the SNR improves by 5.44 dB on average, with a small but consistent advantage over spectral subtraction and no perceptual or intelligibility gain over it. The firmware matches the Python reference to within 1.7 steps of 16-bit audio. Future work is a musical-noise term in the tuning objective, a detector for transient noise, longer recordings with more pauses, and a version of the link over Wi-Fi.

Chapter 7
References

[1] S. F. Boll, “Suppression of acoustic noise in speech using spectral subtraction,” IEEE Trans. Acoust., Speech, Signal Process., vol. 27, no. 2, pp. 113-120, 1979.

[2] Y. Ephraim and D. Malah, “Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator,” IEEE Trans. Acoust., Speech, Signal Process., vol. 32, no. 6, pp. 1109-1121, 1984.

[3] R. Martin, “Noise power spectral density estimation based on optimal smoothing and minimum statistics,” IEEE Trans. Speech Audio Process., vol. 9, no. 5, pp. 504-512, 2001.

[4] I. Cohen and B. Berdugo, “Noise estimation by minima controlled recursive averaging for robust speech enhancement,” IEEE Signal Process. Lett., vol. 9, no. 1, pp. 12-15, 2002.

[5] D. W. Griffin and J. S. Lim, “Signal estimation from modified short-time Fourier transform,” IEEE Trans. Acoust., Speech, Signal Process., vol. 32, no. 2, pp. 236-243, 1984.

[6] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” Math. Comput., vol. 19, no. 90, pp. 297-301, 1965.

[7] P. D. Welch, “The use of fast Fourier transform for the estimation of power spectra,” IEEE Trans. Audio Electroacoust., vol. 15, no. 2, pp. 70-73, 1967.

[8] ITU-T Recommendation P.862.2, “Wideband extension to Recommendation P.862 for the assessment of wideband telephone networks and speech codecs,” 2007.

[9] V. Panayotov, G. Chen, D. Povey and S. Khudanpur, “Librispeech: an ASR corpus based on public domain audio books,” in Proc. IEEE ICASSP, 2015, pp. 5206-5210.

[10] J. Thiemann, N. Ito and E. Vincent, “The diverse environments multi-channel acoustic noise database (DEMAND),” in Proc. Meetings on Acoustics, vol. 19, 2013.

[11] C. H. Taal, R. C. Hendriks, R. Heusdens and J. Jensen, “An algorithm for intelligibility prediction of time-frequency weighted noisy speech,” IEEE Trans. Audio, Speech, Lang. Process., vol. 19, no. 7, pp. 2125-2136, 2011.

[12] S. Holm, “A simple sequentially rejective multiple test procedure,” Scand. J. Statist., vol. 6, no. 2, pp. 65-70, 1979.